COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 6, 2026	**DATE SUBMITTED:**	Oct 6, 2026
NAME:	ANDREW DOLOSO JR. RALPH FLORENZ DUE?AS	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11
BASIC ALGORITHM ANALYSIS

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis.

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

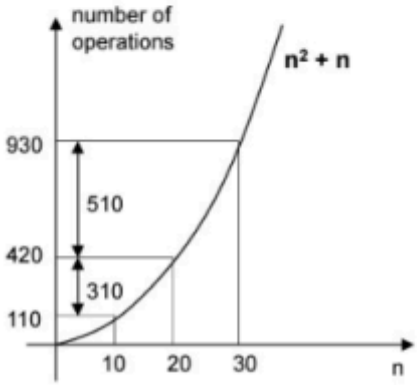
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: _____

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis: Based on our observations the graph shows, the size of the array increases, the total number of operations grows extremely fast. The code running time depends on the array size because that size determines how many total loops the program must perform. The code checks whether the specific element we are searching for exists inside the given array. If that element is not found, the program outputs -1, which leads directly to the worst-case scenario. In this worst-case, the runtime reaches On^2 operations because the outer loop runs N times and the search scans M elements every single time, leading to an M x N calculation or simply On^2.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis.

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	**1 microseconds**	`Time taken to search = 1 microseconds` `Student (18 351) needs vaccination.`	Searching 1,000 sorted elements takes negligible time (1 µs) as binary search requires at most 10 comparison steps.
10,000	**1 microseconds**	`Time taken to search = 1 microseconds` `Student (18 351) needs vaccination.`	Increasing input size by 10x adds only around 3 to 4 extra comparison steps (⌈log_2(1000)⌉ = 14), keeping execution time around 1 µs.
100,000	**2 microseconds**	`Time taken to search = 2 microseconds` `Student (18 351) needs vaccination.`	100,000 elements requires up to 17 comparisons, demonstrating O(log N) logarithmic time complexity where execution speed remains essentially unchanged (2 µs) despite substantial data growth.

Analysis: These results show that binary search has an O(log N) time complexity. Even when the input size increases 100 times, from 1,000 to 100,000 items, the maximum number of comparisons only increases from 10 to 17. Since these extra comparisons take very little CPU time, the measured search time using std::chrono stays almost the same, at around 1 to 2 microseconds.

Output:

```
Time taken to search = 1 microseconds
Student (18 351) needs vaccination.
Time taken to search = 1 microseconds
Student (18 351) needs vaccination.
Time taken to search = 2 microseconds
Student (18 351) needs vaccination.
```

Supplementary Activity:

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Worst-Case Time Complexity : $O(n^2)$
Best-Case Time Complexity : $O(1)$
Space Complexity : $O(1)$

```cpp
#include <iostream>
#include <iomanip>
#include <vector>
#include <chrono>

bool unique(const std::vector<int>& A, long long& comparisons) {
    for (size_t i = 0; i < A.size(); ++i) {
        for (size_t j = i + 1; j < A.size(); ++j) {
            comparisons++;
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}

std::vector<int> makeWorstCase(int n) {
    std::vector<int> worst_case(n);
    for (int i = 0; i < n; ++i) {
        worst_case[i] = i;
    }
    return worst_case;
}
```

```cpp
std::vector<int> makeWorstCase(int n) {
    std::vector<int> worst_case(n);
    for (int i = 0; i < n; ++i) {
        worst_case[i] = i;
    }
    return worst_case;
}

auto start = std::chrono::high_resolution_clock::now();
auto end = std::chrono::high_resolution_clock::now();
double ms = std::chrono::duration<double, std::milli>(end - start).count();

int main() {
    std::vector<int> sizes = {100, 1000, 10000, 20000};

    std::cout << std::setw(8)  << "n"
              << std::setw(16) << "comparisons"
              << std::setw(16) << "n(n-1)/2"
              << std::setw(14) << "time(ms)"
              << std::setw(10) << "unique?" << "\n";

    for (size_t i = 0; i < sizes.size(); ++i) {
        int n = sizes[i];
        std::vector<int> worst_case = makeWorstCase(n);
        long long comparisons = 0;

        start = std::chrono::high_resolution_clock::now();
        bool is_unique = unique(worst_case, comparisons);
        end = std::chrono::high_resolution_clock::now();
        ms = std::chrono::duration<double, std::milli>(end - start).count();
        long long expected = (long long)n * (n - 1) / 2;

        std::cout << std::setw(8)  << n
                  << std::setw(16) << comparisons
                  << std::setw(16) << expected
                  << std::setw(14) << std::fixed << std::setprecision(4) << ms
                  << std::setw(10) << is_unique << "\n";
    }
    return 0;
}
```

n	comparisons	n(n-1)/2	time(ms)	unique?
100	4950	4950	0.0528	1
1000	499500	499500	5.4074	1
10000	49995000	49995000	722.7002	1
20000	199990000	199990000	2606.2060	1

Analysis: In the code, the theoretical and real test results are nearly identical to each other. This is because theoretical analysis predicts that, in the worst-case scenario, the number of comparisons done by Unique(A) will be $n(n-1)/2$, which has a time complexity of $O(n^2)$. The data shows that as the input size increases, the total count of comparisons grows very quickly. Furthermore, the actual execution time follows this same pattern; making n ten times larger increases the running time by 93–98 times, and doubling n increases it by roughly 4.5 times. Therefore, the theory completely explains the real experimental data.

EXPERIMENTAL ANALYSIS

n	comparisons	n(n-1)/2	time(ms)	unique?
100	4950	4950	0.0528	1
1000	499500	499500	5.4074	1
10000	49995000	49995000	722.7002	1
20000	199990000	199990000	2606.2060	1

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

```cpp
#include <iostream>
#include <chrono>
#include <iomanip>

// Linear Recursive Power O(n)
int rpower(int x, int n) {
    if (n == 0) return 1;
    return x * rpower(x, n - 1);
}

// Binary Recursive Power O(Log n)
int brpower(int x, int n) {
    if (n == 0) return 1;
    if (n % 2 != 0) {
        int y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        int y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    int x = 1; // x = 1 prevents integer overflow during runtime testing
    int exponents[] = {100, 1000, 5000, 10000, 20000};
    const int iterations = 5000; // Repeat to get a measurable average time

    // Print headers with fixed column widths
    std::cout << std::left
              << std::setw(15) << "Exponent (n)"
              << std::setw(20) << "rpower Avg (ns)"
              << std::setw(20) << "brpower Avg (ns)" << "\n";
    std::cout << "_____\n";
```

```cpp
    for (int n : exponents) {
        // Measure rpower execution time over multiple iterations
        volatile int sink1 = 0;
        auto start1 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < iterations; ++i) {
            sink1 = rpower(x, n);
        }
        auto end1 = std::chrono::high_resolution_clock::now();
        auto t_rpower = std::chrono::duration_cast<std::chrono::nanoseconds>(end1 - start1).count() / iterations;

        // Measure brpower execution time over multiple iterations
        volatile int sink2 = 0;
        auto start2 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < iterations; ++i) {
            sink2 = brpower(x, n);
        }
        auto end2 = std::chrono::high_resolution_clock::now();
        auto t_brpower = std::chrono::duration_cast<std::chrono::nanoseconds>(end2 - start2).count() / iterations;

        // Print formatted row matching fixed widths
        std::cout << std::left
                  << std::setw(15) << n << "| "
                  << std::setw(20) << t_rpower<< "| "
                  << std::setw(20) << t_brpower << "|\n";
    }

    return 0;
}
```

Output:

```
PS C:\Users\TIPQC\Documents\CPE010_DOLOSO> cd "c:\Users\TIPQC\Documen
Exponent (n)   rpower Avg (ns)     brpower Avg (ns)
_____________________________________________________________
100            | 312                | 27                 |
1000           | 3013               | 48                 |
5000           | 15905              | 60                 |
10000          | 34686              | 75                 |
20000          | 72243              | 70                 |
PS C:\Users\TIPQC\Documents\CPE010_DOLOSO\basicAlgoAnalysis>
```

Analysis:

- This analysis shows that linear recursive power (rpower) needs n recursive calls and multiplications, so it has O(n) time and space complexity. The experiment also confirms that its execution time increases as the exponent increases. On the other hand, binary recursive power (brpower) divides the exponent by half at each step, giving it O(log n) time and space complexity. Because of this, brpower runs much faster, taking less than 1 microsecond for all tested exponents. Overall, binary exponentiation is much faster and uses less stack memory than linear recursion

CONCLUSION, LEARNINGS, AND ANALYSIS

Duenas :

- In this HOA, I saw how the size of the input directly affects how many steps an algorithm takes. I learned how to recognize different types of growth, such as constant, linear, logarithmic, and quadratic. Through this task, I realized that some algorithms handle large amounts of data very well, while others do not. Finally, I learned that checking best-case, average-case, and worst-case scenarios is a great way to judge an algorithm.

Doloso :

- In this activity, I learn a deeper understanding on both theoretical and experimental algorithm analysis. I also learned how theoretical models like Big-O notation accurately predict how runtime scales with input size, which was directly validated by our timing experiments using std::chrono. Comparing linear and logarithmic algorithms clearly demonstrated how algorithmic efficiency and proper design choices improve computational performance on larger datasets.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.